

# VELORA — Module Subscription (Part 1)

Migrations · Enums · Models · Repository

## Folder Structure

```
app/Modules/Subscription/  
├── Controllers/  
│   ├── SubscriptionController.php  
│   └── PlanController.php  
├── DTOs/  
│   └── CreateSubscriptionDTO.php  
├── Enums/  
│   ├── SubscriptionStatus.php  
│   ├── BillingCycle.php  
│   └── BillingMode.php  
├── Events/  
│   ├── SubscriptionActivated.php  
│   ├── SubscriptionExpired.php  
│   └── TrialStarted.php  
├── Listeners/  
│   ├── UnlockFeaturesOnActivation.php  
│   └── LockFeaturesOnExpiry.php  
├── Models/  
│   ├── Plan.php  
│   └── Subscription.php  
├── Repositories/  
│   ├── Contracts/  
│   │   ├── PlanRepositoryInterface.php  
│   │   └── SubscriptionRepositoryInterface.php  
│   └── Eloquent/  
│       ├── PlanRepository.php  
│       └── SubscriptionRepository.php  
├── Requests/  
│   └── SubscribeRequest.php  
├── Resources/  
│   ├── PlanResource.php  
│   └── SubscriptionResource.php  
├── Services/  
│   └── SubscriptionService.php  
├── Commands/  
│   └── CheckExpiredSubscriptions.php  
└── Routes/  
    └── api.php
```

# 1. Migrations

## Plans Table

```
<?php
// database/migrations/subscription/2026_05_19_000010_create_plans_table.php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('plans', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->string('name', 50); // Starter, Growth, Enterprise
            $table->string('slug', 60)->unique(); // starter, growth, enterprise
            $table->text('description')->nullable();
            $table->boolean('is_active')->default(true)->index();

            // Pricing
            $table->bigInteger('price_monthly'); // IDR, smallest unit
            $table->bigInteger('price_yearly');
            $table->integer('trial_days')->default(14);

            // Feature limits (-1 = unlimited)
            $table->integer('max_branches')->default(1);
            $table->integer('max_products')->default(100);
            $table->integer('max_employees')->default(5);
            $table->integer('max_customers')->default(500);
            $table->integer('max_transactions_per_month')->default(500);

            // Feature flags
            $table->boolean('has_ai')->default(false);
            $table->boolean('has_multi_currency')->default(false);
            $table->boolean('has_report_export')->default(false);
            $table->boolean('has_api_access')->default(false);
            $table->boolean('has_barcode_scanner')->default(true);
            $table->boolean('has_purchase_order')->default(false);
            $table->boolean('has_crm')->default(false);

            $table->integer('sort_order')->default(0);
            $table->timestamps();

            $table->index(['is_active', 'sort_order']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('plans');
    }
}
```



# Subscriptions Table

```
<?php
// database/migrations/subscription/2026_05_19_000011_create_subscriptions_table.php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('subscriptions', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->uuid('business_id');
            $table->uuid('plan_id');

            $table->string('status', 20)->default('trial')->index();
            $table->string('billing_cycle', 10)->default('monthly'); // monthly, yearly
            $table->string('billing_mode', 20)->default('manual'); // manual, auto

            // Dates
            $table->timestamp('trial_ends_at')->nullable();
            $table->timestamp('starts_at')->nullable();
            $table->timestamp('ends_at')->nullable();
            $table->timestamp('grace_ends_at')->nullable();
            $table->timestamp('cancelled_at')->nullable();

            // Pricing snapshot (saat subscribe)
            $table->bigInteger('price_paid')->default(0);
            $table->string('currency', 3)->default('IDR');

            // Midtrans
            $table->string('midtrans_order_id')->nullable()->index();
            $table->string('midtrans_transaction_id')->nullable();
            $table->string('midtrans_payment_type')->nullable();

            $table->json('meta')->nullable();
            $table->timestamps();

            $table->foreign('business_id')->references('id')->on('businesses')->cascadeOnDelete();
            $table->foreign('plan_id')->references('id')->on('plans');

            $table->index(['business_id', 'status']);
            $table->index(['ends_at', 'status']);
            $table->index(['grace_ends_at', 'status']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('subscriptions');
    }
};
```

# Subscription Invoices Table

```
<?php
// database/migrations/subscription/2026_05_19_000012_create_subscription_invoices_table.php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('subscription_invoices', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->uuid('subscription_id');
            $table->uuid('business_id');

            $table->string('invoice_number', 50)->unique();
            $table->string('status', 20)->default('unpaid'); // unpaid, paid, cancelled
            $table->bigInteger('amount');
            $table->string('currency', 3)->default('IDR');
            $table->string('billing_cycle', 10);

            // Payment proof (for manual billing)
            $table->string('payment_proof')->nullable();
            $table->timestamp('paid_at')->nullable();
            $table->text('notes')->nullable();

            // Midtrans
            $table->string('midtrans_snap_token')->nullable();
            $table->string('midtrans_redirect_url')->nullable();

            $table->timestamp('due_at');
            $table->timestamps();

            $table->foreign('subscription_id')->references('id')->on('subscriptions');
            $table->foreign('business_id')->references('id')->on('businesses');
            $table->index(['business_id', 'status']);
            $table->index(['due_at', 'status']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('subscription_invoices');
    }
};
```

## 2. Enums

app/Modules/Subscription/Enums/SubscriptionStatus.php

```
<?php

namespace App\Modules\Subscription\Enums;

enum SubscriptionStatus: string
{
    case Trial      = 'trial';
    case Active     = 'active';
    case Grace      = 'grace';
    case Expired    = 'expired';
    case Cancelled  = 'cancelled';
    case Suspended  = 'suspended';

    public function label(): string
    {
        return match ($this) {
            self::Trial      => 'Trial',
            self::Active     => 'Aktif',
            self::Grace       => 'Masa Tenggang',
            self::Expired     => 'Kadaluarsa',
            self::Cancelled   => 'Dibatalkan',
            self::Suspended  => 'Ditangguhkan',
        };
    }

    public function isOperational(): bool
    {
        return in_array($this, [self::Trial, self::Active, self::Grace]);
    }

    public function isExpiredOrCancelled(): bool
    {
        return in_array($this, [self::Expired, self::Cancelled, self::Suspended]);
    }
}
```

## app/Modules/Subscription/Enums/BillingCycle.php

```
<?php

namespace App\Modules\Subscription\Enums;

enum BillingCycle: string
{
    case Monthly = 'monthly';
    case Yearly  = 'yearly';

    public function months(): int
    {
        return match ($this) {
            self::Monthly => 1,
            self::Yearly  => 12,
        };
    }

    public function label(): string
    {
        return match ($this) {
            self::Monthly => 'Bulanan',
            self::Yearly  => 'Tahunan',
        };
    }
}
```

## app/Modules/Subscription/Enums/BillingMode.php

```
<?php

namespace App\Modules\Subscription\Enums;

enum BillingMode: string
{
    case Manual = 'manual';
    case Auto   = 'auto';

    public function label(): string
    {
        return match ($this) {
            self::Manual => 'Transfer Manual',
            self::Auto   => 'Auto-Charge (Midtrans)',
        };
    }
}
```

### 3. Models

#### app/Modules/Subscription/Models/Plan.php

```
<?php

namespace App\Modules\Subscription\Models;

use App\Shared\Traits\HasUuid;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Plan extends Model
{
    use HasUuid;

    protected $fillable = [
        'id', 'name', 'slug', 'description', 'is_active',
        'price_monthly', 'price_yearly', 'trial_days',
        'max_branches', 'max_products', 'max_employees',
        'max_customers', 'max_transactions_per_month',
        'has_ai', 'has_multi_currency', 'has_report_export',
        'has_api_access', 'has_barcode_scanner',
        'has_purchase_order', 'has_crm', 'sort_order',
    ];

    protected function casts(): array
    {
        return [
            'is_active'          => 'boolean',
            'has_ai'             => 'boolean',
            'has_multi_currency' => 'boolean',
            'has_report_export'  => 'boolean',
            'has_api_access'     => 'boolean',
            'has_barcode_scanner' => 'boolean',
            'has_purchase_order' => 'boolean',
            'has_crm'            => 'boolean',
        ];
    }

    public function subscriptions(): HasMany
    {
        return $this->hasMany(Subscription::class, 'plan_id');
    }

    public function isUnlimited(string $feature): bool
    {
        return $this->{$feature} === -1;
    }

    public function scopeActive($query): \Illuminate\Database\Eloquent\Builder
    {
        return $query->where('is_active', true)->orderBy('sort_order');
    }
}
```



## app/Modules/Subscription/Models/Subscription.php

```
<?php

namespace App\Modules\Subscription\Models;

use App\Modules\Subscription\Enums\BillingCycle;
use App\Modules\Subscription\Enums\BillingMode;
use App\Modules\Subscription\Enums\SubscriptionStatus;
use App\Modules\Tenant\Models\Business;
use App\Shared\Traits\HasUuid;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Subscription extends Model
{
    use HasUuid;

    protected $fillable = [
        'id', 'business_id', 'plan_id', 'status',
        'billing_cycle', 'billing_mode',
        'trial_ends_at', 'starts_at', 'ends_at',
        'grace_ends_at', 'cancelled_at',
        'price_paid', 'currency',
        'midtrans_order_id', 'midtrans_transaction_id',
        'midtrans_payment_type', 'meta',
    ];

    protected function casts(): array
    {
        return [
            'status'      => SubscriptionStatus::class,
            'billing_cycle' => BillingCycle::class,
            'billing_mode' => BillingMode::class,
            'trial_ends_at' => 'datetime',
            'starts_at'    => 'datetime',
            'ends_at'      => 'datetime',
            'grace_ends_at' => 'datetime',
            'cancelled_at' => 'datetime',
            'meta'         => 'array',
        ];
    }

    /* — Relations — */

    public function business(): BelongsTo
    {
        return $this->belongsTo(Business::class, 'business_id');
    }

    public function plan(): BelongsTo
    {
        return $this->belongsTo(Plan::class, 'plan_id');
    }
}
```

```

public function invoices(): HasMany
{
    return $this->hasMany(SubscriptionInvoice::class, 'subscription_id');
}

/* — Helpers — */

public function isActive(): bool
{
    return $this->status->isOperational();
}

public function isExpired(): bool
{
    return $this->status->isExpiredOrCancelled();
}

public function isInTrial(): bool
{
    return $this->status === SubscriptionStatus::Trial
        && $this->trial_ends_at->isFuture();
}

public function isInGrace(): bool
{
    return $this->status === SubscriptionStatus::Grace
        && $this->grace_ends_at->isFuture();
}

public function daysUntilExpiry(): int
{
    $date = $this->ends_at ?? $this->trial_ends_at;
    return $date ? (int) now()->diffInDays($date, false) : 0;
}

public function canUseFeature(string $feature): bool
{
    return $this->isActive() && ($this->plan->{$feature} ?? false);
}

public function isWithinLimit(string $limitField, int $currentCount): bool
{
    $limit = $this->plan->{$limitField} ?? 0;
    return $limit === -1 || $currentCount < $limit;
}
}

```

## 4. Repository Interfaces

**app/Modules/Subscription/Repositories/Contracts/PlanRepositoryInterface.php**

```
<?php

namespace App\Modules\Subscription\Repositories\Contracts;

use App\Modules\Subscription\Models\Plan;
use Illuminate\Database\Eloquent\Collection;

interface PlanRepositoryInterface
{
    public function findById(string $id): ?Plan;
    public function findBySlug(string $slug): ?Plan;
    public function allActive(): Collection;
}
```

**app/Modules/Subscription/Repositories/Contracts/SubscriptionRepositoryInterface.php**

```
<?php

namespace App\Modules\Subscription\Repositories\Contracts;

use App\Modules\Subscription\Models\Subscription;

interface SubscriptionRepositoryInterface
{
    public function findById(string $id): ?Subscription;
    public function findByBusiness(string $businessId): ?Subscription;
    public function findExpiredActive(): \Illuminate\Database\Eloquent\Collection;
    public function findExpiredGrace(): \Illuminate\Database\Eloquent\Collection;
    public function create(array $data): Subscription;
    public function update(string $id, array $data): Subscription;
}
```

## 5. Repository Implementations

app/Modules/Subscription/Repositories/Eloquent/PlanRepository.php

```
<?php

namespace App\Modules\Subscription\Repositories\Eloquent;

use App\Modules\Subscription\Models\Plan;
use App\Modules\Subscription\Repositories\Contracts\PlanRepositoryInterface;
use Illuminate\Database\Eloquent\Collection;

class PlanRepository implements PlanRepositoryInterface
{
    public function findById(string $id): ?Plan
    {
        return Plan::find($id);
    }

    public function findBySlug(string $slug): ?Plan
    {
        return Plan::where('slug', $slug)->first();
    }

    public function allActive(): Collection
    {
        return Plan::active()->get();
    }
}
```

## app/Modules/Subscription/Repositories/Eloquent/SubscriptionRepository.php

```
<?php

namespace App\Modules\Subscription\Repositories\Eloquent;

use App\Modules\Subscription\Models\Subscription;
use App\Modules\Subscription\Repositories\Contracts\SubscriptionRepositoryInterface;
use Illuminate\Database\Eloquent\Collection;

class SubscriptionRepository implements SubscriptionRepositoryInterface
{
    public function findById(string $id): ?Subscription
    {
        return Subscription::with(['plan', 'business'])->find($id);
    }

    public function findByBusiness(string $businessId): ?Subscription
    {
        return Subscription::where('business_id', $businessId)
            ->with('plan')
            ->latest()
            ->first();
    }

    public function findExpiredActive(): Collection
    {
        return Subscription::where('status', 'active')
            ->where('ends_at', '<=', now())
            ->with('business')
            ->get();
    }

    public function findExpiredGrace(): Collection
    {
        return Subscription::where('status', 'grace')
            ->where('grace_ends_at', '<=', now())
            ->with('business')
            ->get();
    }

    public function create(array $data): Subscription
    {
        return Subscription::create($data);
    }

    public function update(string $id, array $data): Subscription
    {
        $sub = Subscription::findOrFail($id);
        $sub->update($data);
        return $sub->fresh(['plan', 'business']);
    }
}
```

## 6. Update RepositoryServiceProvider

```
// Tambahkan ke RepositoryServiceProvider::register()

use App\Modules\Subscription\Repositories\Contracts\PlanRepositoryInterface;
use App\Modules\Subscription\Repositories\Eloquent\PlanRepository;
use App\Modules\Subscription\Repositories\Contracts\SubscriptionRepositoryInterface;
use App\Modules\Subscription\Repositories\Eloquent\SubscriptionRepository;

$this->app->bind(PlanRepositoryInterface::class, PlanRepository::class);
$this->app->bind(SubscriptionRepositoryInterface::class, SubscriptionRepository::class);
```

## 7. Update Business Model — hasActiveSubscription()

```
// app/Modules/Tenant/Models/Business.php

public function subscription(): \Illuminate\Database\Eloquent\Relations\HasOne
{
    return $this->hasOne(\App\Modules\Subscription\Models\Subscription::class, 'business_id')
        ->latest();
}

public function hasActiveSubscription(): bool
{
    return $this->subscription?->isActive() ?? false;
}

public function currentPlan(): ?\App\Modules\Subscription\Models\Plan
{
    return $this->subscription?->plan;
}
```